



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Всех приветствую! В этом уроке мы поговорим про потоки данных используемые в Linux, в нашем случае в Kali Linux.

Каждая программа, запущенная из командной строки, имеет три подключенных к ней потока данных. Эти потоки определены следующим образом:

Имя потока:

Standart Input (STDIN) - Данные поступающие в программу (номер потока 0)

Standart Output (STDOUT) - Вывод данных из программы (например, в Терминал) (номер потока 1)

Standart Error (STDERR) - Сообщения об ошибках (номер потока 2) (которые мы очень часто встречаем в Терминале)

Передача осуществляется с помощью оператора | (мы его использовали в одном из предыдущих уроков, когда передавали данные на ввод команде `grep`) и перенаправление с помощью операторов `>` и `<`. Таким образом мы можем соединять потоки между программами и файлами для обеспечения практически бесконечного числа возможных вариантов использования.

Перенаправление в новый файл

В предыдущих примерах команд (уроках) вывод был напечатан на экран. Это удобно в большинстве случаев, но мы можем использовать оператор `>` для сохранения вывода в файл, чтобы сохранить его для дальнейшего использования:

```
kali@kali:~$ echo "test" > test1.txt
kali@kali:~$ ls
Desktop Documents Downloads Music Pictures Public test1.txt Template
kali@kali:~$ cat test1.txt
test
```

Соответственно если мы перенаправим вывод в несуществующий файл, файл будет создан автоматически. Однако если мы сохраним вывод в уже существующий файл, содержимое этого файла будет заменено. Будьте осторожны с перенаправлением! Функция отмены не существует!

Перенаправление в существующий файл

Для добавления дополнительных данных в существующий файл (в отличие от перезаписи файла) используйте оператор >>:

```
kali@kali:~$ echo "123456" >> test1.txt
```

Перенаправление из файла

В следующем примере мы перенаправляем данные из файла в команду wc. Попробуем сделать это с помощью команды wc -m, которая подсчитывает символы в файле:

```
kali@kali:~$ wc -m < test1.txt  
12
```

Перенаправление сообщений об ошибках

Чтобы лучше понять, как это работает, рассмотрим пример, который перенаправляет стандартную ошибку (STDERR):

```
kali@kali:~$ ls ./test  
ls: cannot access './test': No such file or directory  
kali@kali:~$ ls ./test 2>error.txt  
kali@kali:~$ cat error.txt  
ls: cannot access './test': No such file or directory
```

Обратите внимание, что запись в файле "error.txt" содержит только сообщение об ошибке. Мы сделали это, добавив номер потока (2) к оператору ">" (2=STDERR).

Также можно использовать конструкцию вида "2>&1", которая перенаправляет stderr в стандартный вывод. Использование 2> и 1 полезно для обнаружения и обработки ошибок в сценариях Bash. Перенаправляя stderr на stdout, можно фиксировать как обычный вывод, так и вывод ошибок в одном потоке, что упрощает анализ и обработку ошибок. Это может быть особенно полезно в сценариях, требующих обработки ошибок, где нужно знать, успешно ли выполнена команда или произошла ошибка. Например:

```
(kali@kali)-[~/Desktop]
$ ls ~/Desktop >> result.txt 2>&1

(kali@kali)-[~/Desktop]
$ ls ~/Desktop2 >> result.txt 2>&1
```

Содержимое файла result.txt:

```
~/Desktop/result.txt - Mousepad
File Edit Search View Document Help
1 |123
2
3 blobs
4 Burp_Suite_Professional-2023-10-2_x86_64.AppImage
5
6 exploit172_16hp.py
7 hmac.php
8 ngrok
9 ngrok-v3-stable-linux-amd64.tgz
10 result.txt
11 Serial.php
12
13
14
15 WhatsApp
16 ysoserial-all.jar
17 ls: cannot access '/home/kali/Desktop2': No such file or directory
18
```

Pipe

В Linux пайп (pipe) — это мощный механизм, позволяющий перенаправлять вывод одной команды в качестве ввода для другой команды. Он используется с помощью символа вертикальной черты |. Пайпы играют ключевую роль в философии Unix, где программы должны делать одну вещь хорошо и легко интегрироваться с другими программами. Рассмотрим следующий пример:

```
kali@kali:~$ cat error.txt
ls: cannot access '/test': No such file or directory
kali@kali:~$ cat error.txt | wc -m
54
kali@kali:~$ cat error.txt | wc -m > count.txt
kali@kali:~$ cat count.txt
54
```

Мы использовали символ pipe (|) для перенаправления вывода команды cat на вход команды wc. Эта концепция может показаться тривиальной, но

объединение различных команд в "трубопровод" является мощным инструментом для манипулирования любыми данными.

Пайпы часто используются для обработки текстовых данных. Например, комбинация команд `cat file.txt | sort | uniq -c | sort -nr` сначала выводит содержимое файла, затем сортирует его, удаляет дублирующиеся строки, подсчитывает количество каждой уникальной строки и, наконец, сортирует их по убыванию частоты. Такие цепочки могут значительно упростить выполнение сложных задач обработки данных.

Команды, используемые в уроке:

```
kali@kali:~$ echo "test" > test1.txt
```

```
kali@kali:~$ ls
```

```
kali@kali:~$ cat test1.txt
```

```
kali@kali:~$ echo "123456" >> test1.txt
```

```
kali@kali:~$ wc -m < test1.txt
```

```
kali@kali:~$ ls ./test
```

```
kali@kali:~$ ls ./test 2>error.txt
```

```
kali@kali:~$ cat error.txt
```

```
cat error.txt | wc -m > count.txt
```